

What is MapReduce?

- ❑ MapReduce is a framework using which we can write applications to process huge amounts of data, in parallel, on large clusters of commodity hardware in a reliable manner.
- ❑ MapReduce is a processing technique and a program model for distributed computing based on java.
- ❑ The MapReduce algorithm contains two important tasks, namely Map and Reduce.
- ❑ Map takes a set of data and converts it into another set of data, where individual elements are broken down into tuples (key/value pairs).
- ❑ Secondly, reduce task, which takes the output from a map as an input and combines those data tuples into a smaller set of tuples. As the sequence of the name MapReduce implies, the reduce task is always performed after the map job.

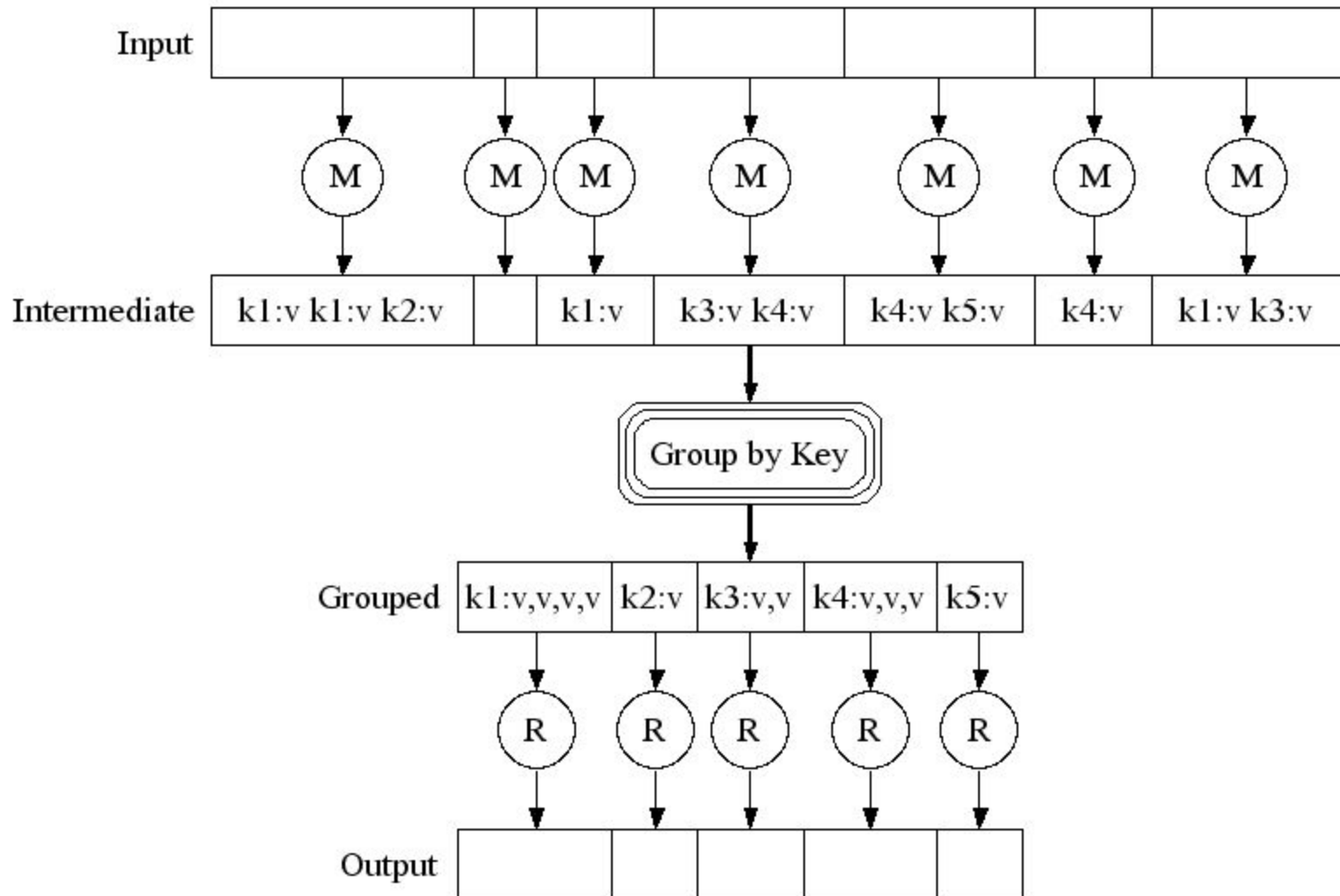
What is MapReduce?

- ❑ The major advantage of MapReduce is that it is easy to scale data processing over multiple computing nodes.
- ❑ Under the MapReduce model, the data processing primitives are called mappers and reducers.
- ❑ Decomposing a data processing application into mappers and reducers is sometimes nontrivial. But, once we write an application in the MapReduce form, scaling the application to run over hundreds, thousands, or even tens of thousands of machines in a cluster is merely a configuration change.
- ❑ This simple scalability is what has attracted many programmers to use the MapReduce model.

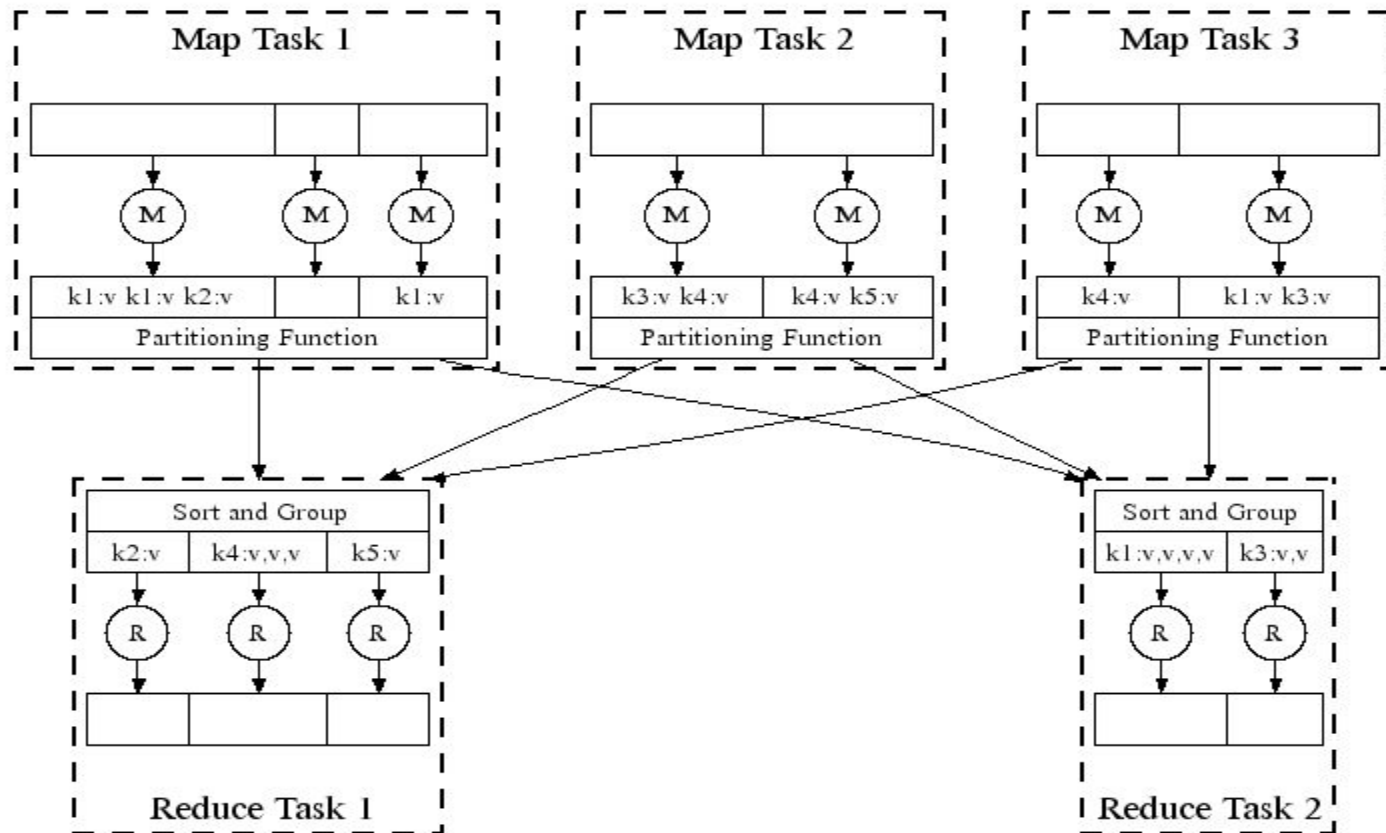
What is MapReduce?

- ❑ MapReduce program executes in three stages, namely map stage, shuffle stage, and reduce stage.
- ❑ Map stage: The map or mapper's job is to process the input data. Generally the input data is in the form of file or directory and is stored in the Hadoop file system (HDFS). The input file is passed to the mapper function line by line. The mapper processes the data and creates several small chunks of data.
- ❑ Reduce stage: This stage is the combination of the Shuffle stage and the Reduce stage. The Reducer's job is to process the data that comes from the mapper. After processing, it produces a new set of output, which will be stored in the HDFS.

The MapReduce Framework



Automatic Parallel Execution



Handles failures automatically, e.g., restarts tasks if a node fails; runs multiples copies of the same task to avoid a slow task slowing down the whole job

MapReduce in Hadoop (1)

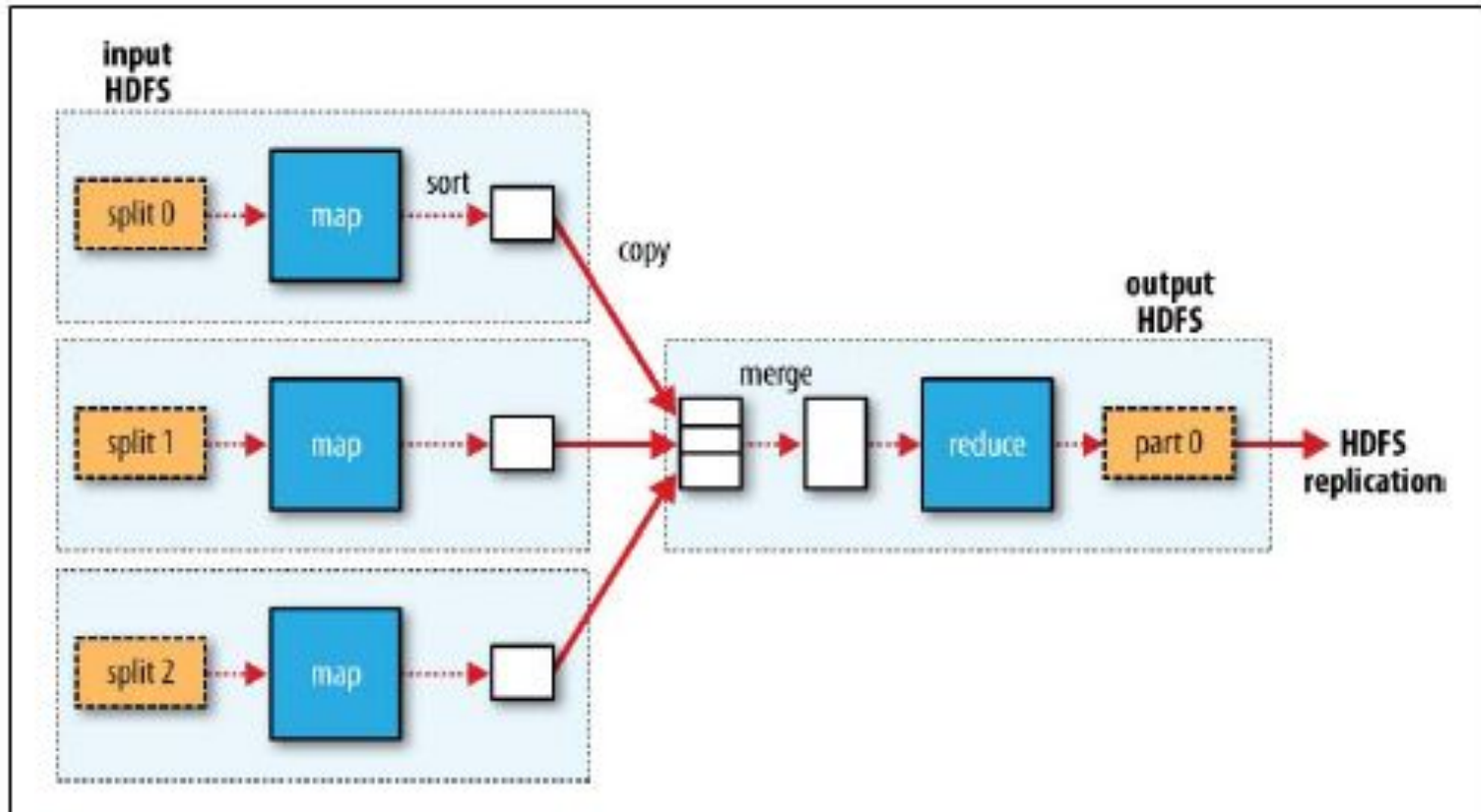


Figure 2-2. MapReduce data flow with a single reduce task

MapReduce in Hadoop (2)

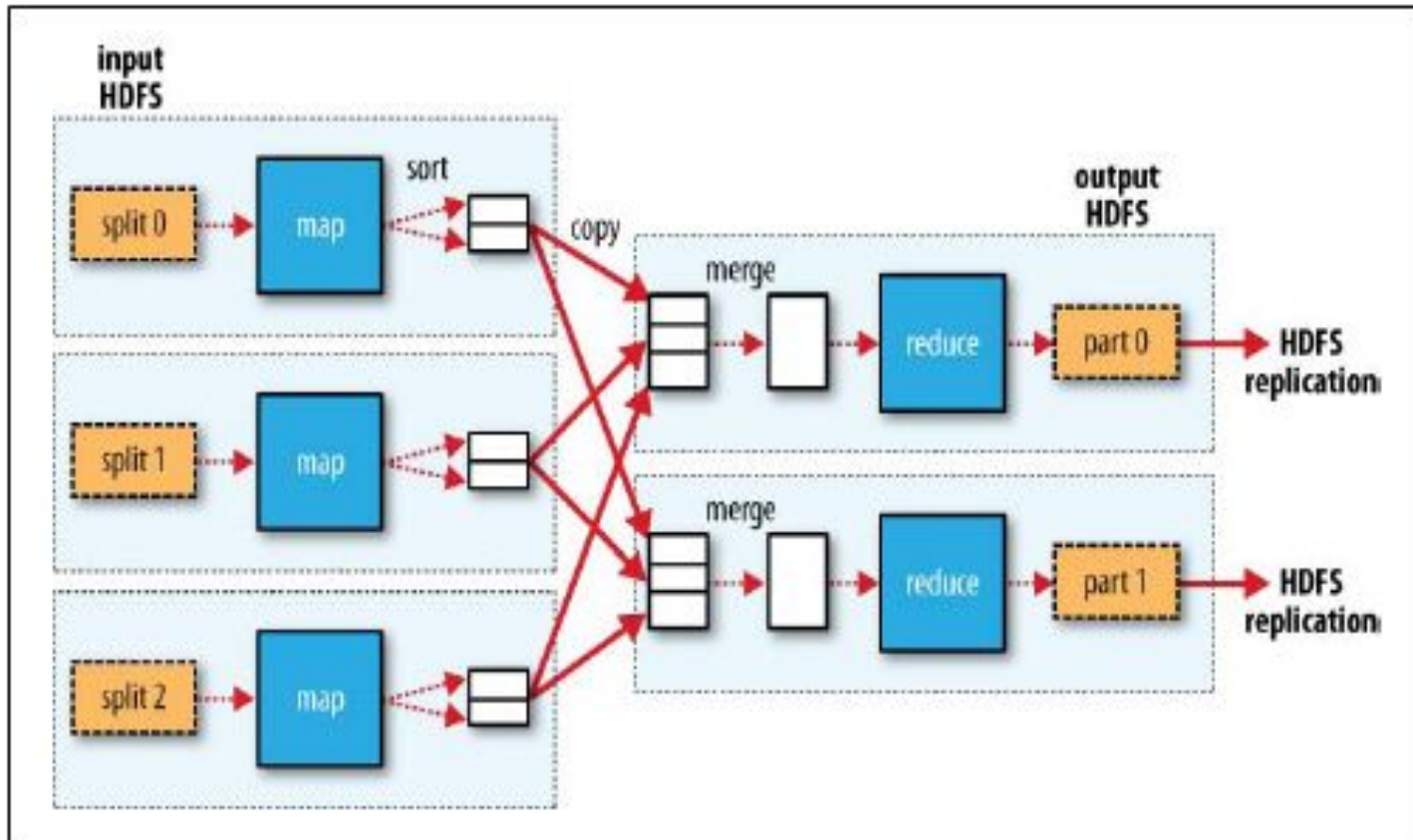
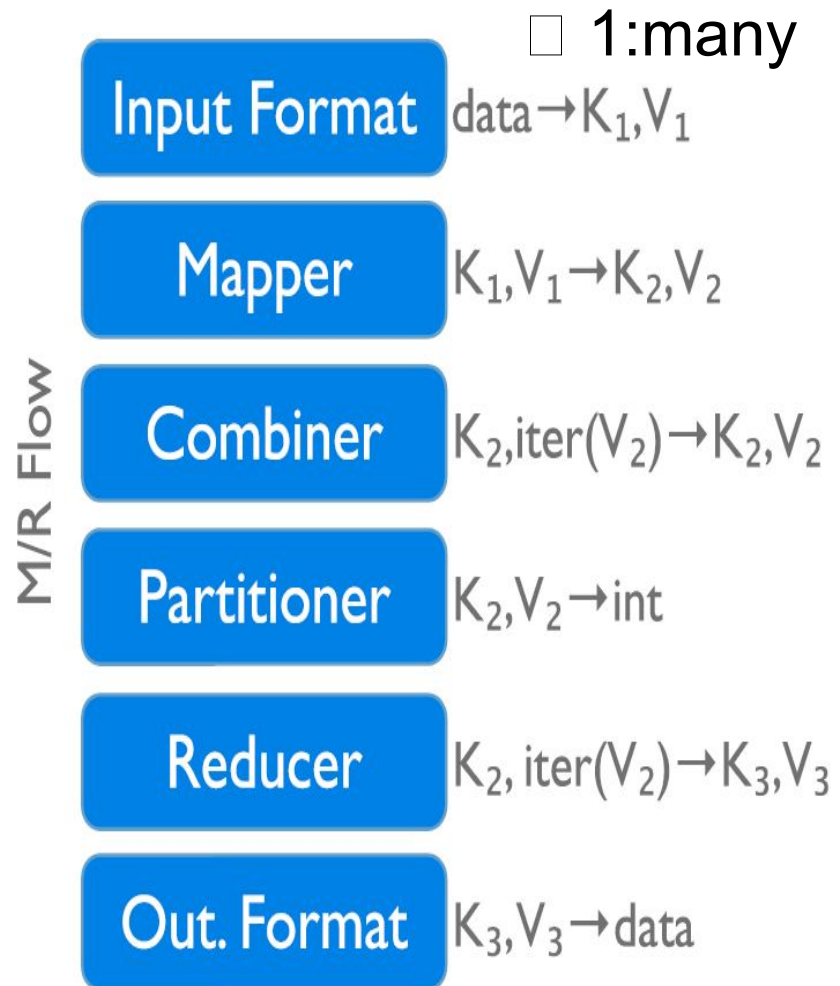


Figure 2-3. MapReduce data flow with multiple reduce tasks

Data Flow in a MapReduce

- InputFormat
- Map function
- Partitioner
- Sorting & Merging
- Combiner
- Shuffling
- Merging
- Reduce function
- OutputFormat



Data Flow in a MapReduce

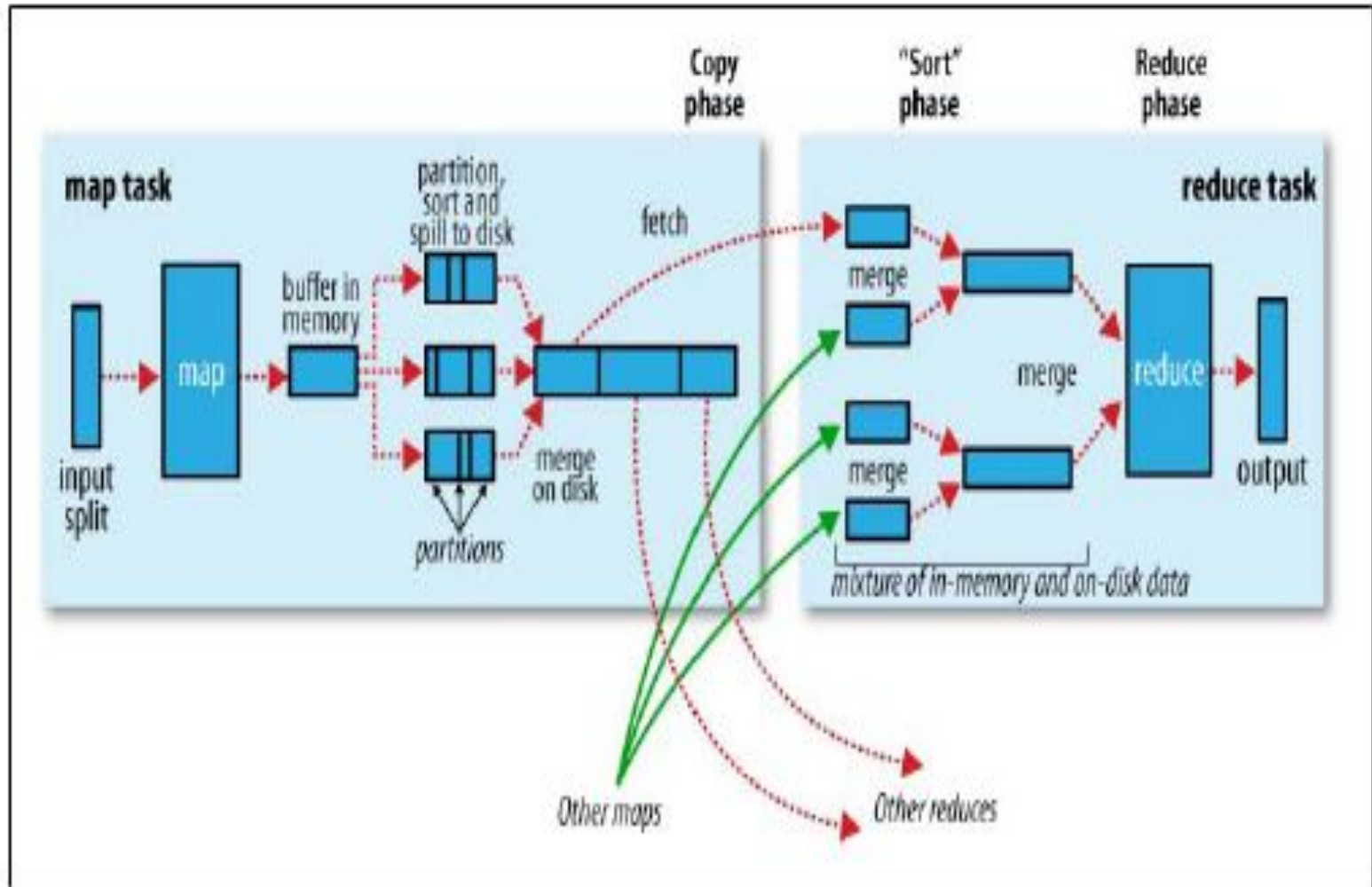


Figure 6-4. Shuffle and sort in MapReduce

Write a program using Hadoop that counts number of occurrences of words in a file.

1 apple orange mango

2 apple plum grapes

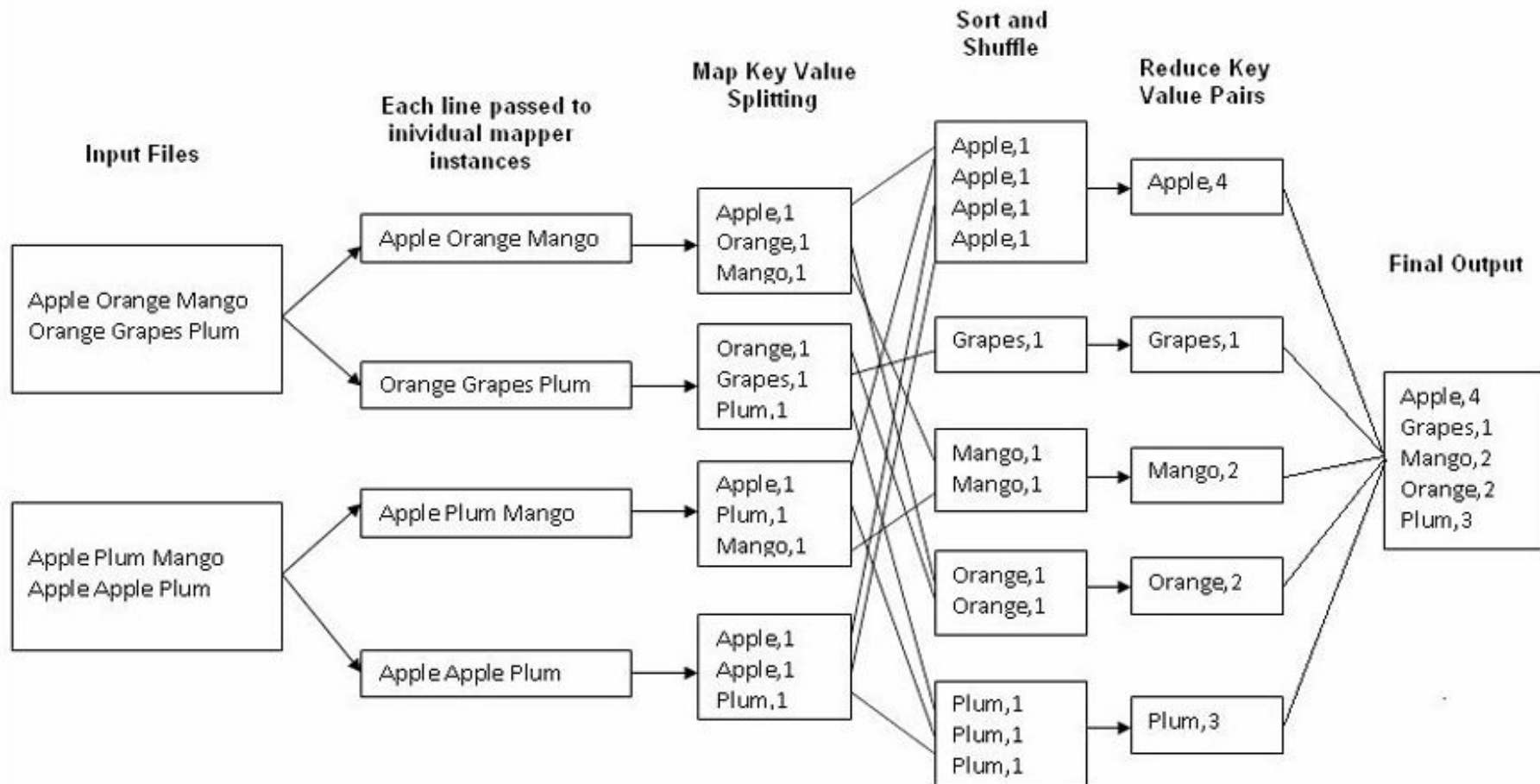
3 pineapple raspberry banana

4 banana orange apple

5 plum raspberry pomgranate

6 grapes grapes apple mango

The MapReduce Framework for Word Count



Terminology

- ❑ **Mapper** - Mapper maps the input **key/value** pairs to a set of intermediate **key/value** pair.
- ❑ **NamedNode** - Node that manages the Hadoop Distributed File System (HDFS).
- ❑ **DataNode** - Node where data is presented in advance before any processing takes place.
- ❑ **MasterNode** - Node where JobTracker runs and which accepts job requests from clients.
- ❑ **SlaveNode** - Node where Map and Reduce program runs.
- ❑ **JobTracker** - Schedules jobs and tracks the assign jobs to Task tracker.
- ❑ **Task Tracker** - Tracks the task and reports status to JobTracker.
- ❑ **Job** - A program is an execution of a Mapper and Reducer across a dataset.
- ❑ **Task** - An execution of a Mapper or a Reducer on a slice of data.

Mapper

```
public class WordCount {  
  
    public static class Map extends Mapper<LongWritable, Text, Text, IntWritable> {  
        private final static IntWritable one = new IntWritable(1);  
        private Text word = new Text();  
  
        public void map(LongWritable key, Text value, Context context)  
            throws IOException, InterruptedException {  
            String line = value.toString();  
            StringTokenizer tokenizer = new StringTokenizer(line);  
            while (tokenizer.hasMoreTokens()) {  
                word.set(tokenizer.nextToken());  
                context.write(word, one);  
            }  
        }  
    }  
}
```

Reducer

```
public static class Reduce extends Reducer<Text, IntWritable, Text, IntWritable> {  
  
    public void reduce(Text key, Iterable<IntWritable> values, Context context)  
        throws IOException, InterruptedException {  
        int sum = 0;  
        for (IntWritable val : values) {  
            sum += val.get();  
        }  
        context.write(key, new IntWritable(sum));  
    }  
}
```


Reducer

```
public static void main(String[] args) throws Exception {  
    Configuration conf = new Configuration();  
  
    Job job = new Job(conf, "wordcount");  
  
    job.setOutputKeyClass(Text.class);  
    job.setOutputValueClass(IntWritable.class);  
  
    job.setMapperClass(Map.class);  
    job.setReducerClass(Reduce.class);  
  
    job.setInputFormatClass(TextInputFormat.class);  
    job.setOutputFormatClass(TextOutputFormat.class);  
  
    FileInputFormat.addInputPath(job, new Path(args[0]));  
    FileOutputFormat.setOutputPath(job, new Path(args[1]));  
  
    job.waitForCompletion(true);  
}
```

Lifecycle of a MapReduce Job

```
File Edit Options Buffers Tools Java Help
public class WordCount {
    public static class Map extends MapReduceBase implements
        Mapper<LongWritable, Text, Text, IntWritable> {
        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();

        public void map(LongWritable key, Text value, OutputCollector<Text, IntWritable>
            output, Reporter reporter) throws IOException {
            String line = value.toString();
            StringTokenizer tokenizer = new StringTokenizer(line);
            while (tokenizer.hasMoreTokens()) {
                word.set(tokenizer.nextToken());
                output.collect(word, one);
            }
        }

        public static class Reduce extends MapReduceBase implements
            Reducer<Text, IntWritable, Text, IntWritable> {
            public void reduce(Text key, Iterator<IntWritable> values, OutputCollector<Text,
                IntWritable> output, Reporter reporter) throws IOException {
                int sum = 0;
                while (values.hasNext()) { sum += values.next().get(); }
                output.collect(key, new IntWritable(sum));
            }

            public static void main(String[] args) throws Exception {
                JobConf conf = new JobConf(WordCount.class);
                conf.setJobName("wordcount");
                conf.setOutputKeyClass(Text.class);
                conf.setOutputValueClass(IntWritable.class);
                conf.setMapperClass(Map.class);
                conf.setCombinerClass(Reduce.class);
                conf.setReducerClass(Reduce.class);
                conf.setInputFormat(TextInputFormat.class);
                conf.setOutputFormat(TextOutputFormat.class);
                FileInputFormat.setInputPaths(conf, new Path(args[0]));
                FileOutputFormat.setOutputPath(conf, new Path(args[1]));

                JobClient.runJob(conf);
            }
        }
    }
}
```

Map function

Reduce function

Run this program as a
MapReduce job

Compilation and Execution

- Let us assume we are in the home directory of a Hadoop user (e.g. /home/StudentName).
- Follow the steps given below to compile and execute the above program.
- **Step 1**
 - The following command is to create a directory to store the compiled java classes.
 - `$ mkdir word`

Compilation and Execution

- **Step 2**

Download [hadoop-core-1.2.1.jar](http://mvnrepository.com/artifact/org.apache.hadoop/hadoop-core/1.2.1), which is used to compile and execute the MapReduce program. Visit the following link

<http://mvnrepository.com/artifact/org.apache.hadoop/hadoop-core/1.2.1>

To download the jar. Let us assume the downloaded folder is /home/rashmi/words.

- **Step 3**

The following commands are used for compiling the wordcount.java program and creating a jar for the program.

```
$ javac -classpath hadoop-core-1.2.1.jar words/WordCount.java
```

```
$ jar -cvf words.jar -C words/ .
```

Compilation and Execution

- **Step 4**

- The following command is used to create an input directory in HDFS.
- `$hadoop fs -mkdir /input`

- **Step 5**

- The following command is used to copy input dataset file on HDFS.
- `$hadoop fs -put fruits.txt /input`

- **Step 6**

- The following command is used to verify the files in the input directory.
- `$hadoop fs -ls /input`

Compilation and Execution

- **Step 7**

- The following command is used to run the Wordcount application by taking the input files from the input directory.
- `$hadoop jar words.jar WordCount /input /output`
- Wait for a while until the file is executed. After execution, the output will contain the number of input splits, the number of Map tasks, the number of reducer tasks, etc. The output directory must *not* be existing already.

Compilation and Execution

- **Step 8**

- The following command is used to verify the resultant files in the output folder.
- `$hadoop fs -ls /output`

- **Step 9**

- The following command is used to see the output in part-r-00000 file. This file is generated by HDFS.
- `$hadoop fs -cat /output/part-r-00000`

Compilation and Execution

- **Step 10**

The following command is used to copy the output file from HDFS to the local file system for analyzing.

- `$hadoop fs -get /output/part-r-00000`

Output

apple	4	
banana	2	
grapes	3	
mango	2	
orange	2	
pineapple		1
plum	2	
pomgranate		1
raspberry		2